

课程笔记

[LLM Agents F24] LLM Agents: History & Overview —Shunyu Yao

五道口纳什 & Codex

2026 年 3 月 28 日



SHUNYU YAO
SENIOR STAFF RESEARCH
SCIENTIST



视频作者/频道: Berkeley RDI

发布日期: Fall 2024

视频时长: 约 75 分钟

视 频 链 接: https://www.youtube.com/playlist?list=PLoROMvodv4r0Y23Y0BoGoBGgQ1zmU_MT_

目录

1 引言：什么是 LLM Agent ?	2
1.1 Agent 的定义	2
1.2 本章小结	2
2 LLM Agent 之前的历史	2
2.1 基于规则的 Text Agent	2
2.2 基于强化学习的 Text Agent	2
2.3 LLM 的革命性变化	3
2.4 本章小结	3
3 推理与行动的融合：ReAct	3
3.1 Reasoning 范式与 Acting 范式	3
3.2 ReAct：推理 + 行动的统一框架	3
3.3 ReAct 的通用性	4
3.4 本章小结	4
4 Reasoning Agent 的理论意义	4
4.1 推理作为无限维的内部动作	4
4.2 三种 Agent 范式的对比	4
4.3 本章小结	5
5 长期记忆 (Long-term Memory)	5
5.1 短期记忆的局限	5
5.2 Reflexion：最简单的长期记忆	5
5.3 更复杂的长期记忆形式	6
5.4 KOALA：Agent 的统一抽象	6
5.5 本章小结	6
6 新兴应用领域	6
6.1 数字自动化 (Digital Automation)	6
6.2 科学发现	6
6.3 本章小结	6
7 未来方向	7
7.1 训练 (Training)	7
7.2 接口 (Interface)	7
7.3 鲁棒性 (Robustness)	7
7.4 人机协作 (Human-in-the-loop)	7
7.5 评测基准 (Benchmarks)	7
7.6 本章小结	7

8 总结与延伸	8
8.1 核心要点	8
8.2 拓展阅读	8

1 引言：什么是 LLM Agent？

Shunyu Yao (姚顺雨) 是 ReAct 框架的提出者，现就职于 OpenAI。本次讲座系统梳理了 LLM Agent 的定义、历史发展脉络和未来方向。

1.1 Agent 的定义

Agent 的三层定义

1. **Text Agent**: 与环境交互的智能系统，其动作 (action) 和观测 (observation) 均以语言 (text) 形式表达。Text Agent 不一定使用 LLM，早在 AI 诞生之初就有基于规则的 Text Agent。
2. **LM Agent**: 使用语言模型来生成动作的 Text Agent。
3. **Reasoning Agent**: 使用语言模型**推理后再行动**的 Agent——这是一个质的飞跃。

定义 Agent 需要回答两个子问题：什么是“智能” (intelligence)? 什么是“环境” (environment)?

- 环境可以是物理的（机器人、自动驾驶）或数字的（视频游戏、网页、代码编辑器）。
- “智能”的定义随时代演变——60 年前简单聊天机器人就算智能，如今 ChatGPT 都不再令人惊讶。

1.2 本章小结

LLM Agent 的核心创新不是让语言模型执行动作，而是让它**先推理再行动**。Reasoning Agent 区别于此前所有 Agent 范式的关键在于：推理 (thinking) 是一种内部动作，具有无限的语言空间。

2 LLM Agent 之前的历史

2.1 基于规则的 Text Agent

ELIZA (1966)

最早的聊天机器人之一，仅用少量规则（不断追问或复述用户所说）就能给人以“智能”的印象。然而，基于规则的方法具有严重的局限性：

- 高度领域特定 (domain-specific)
- 每个新领域需要重新编写规则
- 无法扩展到复杂开放域

2.2 基于强化学习的 Text Agent

在 LLM 出现之前，另一个流行范式是用 RL 构建 Text Agent（如文字游戏 agent）：

- 将文本观测和动作视为视频游戏中的像素和键盘输入
- 使用 RL 优化奖励信号

- **局限**：领域特定、需要精确的奖励信号、训练开销大

2.3 LLM 的革命性变化

LLM 仅通过在海量文本上训练 next-token prediction，就能在推理时通过 prompting 解决各种新任务。这种**通用性**和**少样本学习能力**使其成为构建 Agent 的理想“大脑”。

2.4 本章小结

从基于规则的 symbolic AI agent 到基于 RL 的 deep agent，再到基于 LLM 的 reasoning agent，Agent 的内部表示从符号逻辑到神经嵌入再到自然语言，一步步走向更灵活、更通用。

3 推理与行动的融合：ReAct

3.1 Reasoning 范式与 Acting 范式

在 ReAct 出现之前，LLM 的应用分裂为两个独立的范式：

- **Reasoning**（如 Chain-of-Thought）：LLM 内部逐步推理，增强 test-time compute，但无法获取外部知识或工具。
- **Acting**（如 RAG、tool use）：LLM 调用外部环境（搜索引擎、计算器、API），获取知识和计算能力，但缺乏推理以适应复杂局面。

两个范式各自解决 QA 的不同子问题（推理类问题 vs. 知识类问题 vs. 计算类问题），导致方法碎片化（piecemeal）。

3.2 ReAct：推理 + 行动的统一框架

ReAct 的核心思想

让 LLM 交替生成**思考**（thought）和**动作**（action），观测（observation）由环境返回。只需写一个示例轨迹作为 prompt，展示如何思考和行动来解决任务，LLM 就能模仿这种模式。

具体工作流程：

1. LLM 生成一个 thought（如“我需要查找这三家公司的市值”）
2. LLM 生成一个 action（如 Google[Apple market cap]）
3. 环境返回 observation（搜索结果）
4. 将 thought + action + observation 追加到上下文，LLM 继续生成下一步
5. 重复直到任务完成

推理与行动的双向增益

- **行动帮助推理**：从外部获取实时信息、执行计算
- **推理帮助行动**：根据当前情况规划策略、在异常时重新调整计划。例如，当搜索返回股价而非市值时，推理可以指导下一步查找股份数量来间接计算。

3.3 ReAct 的通用性

ReAct 不局限于问答——任何可以转化为“文本游戏”的任务都可以使用 ReAct 框架：

- 网页浏览、在线购物
- 视频游戏（通过 image captioning 将像素转为文本）
- 家庭任务模拟（ALFWorld）
- 软件工程（SWE-bench）

没有推理的 Acting Agent 的局限

在 ALFWorld 游戏中，纯 acting agent（不生成 thought）在遇到异常时（如目标物体不在预期位置）会反复执行同一动作，无法适应。加入 thinking action 后，agent 可以反思当前状况并重新规划。

3.4 本章小结

ReAct 的贡献不仅是一个具体方法，更是提出了 Reasoning Agent 的范式——将推理作为 Agent 的一种特殊“内部动作”。这种范式系统性地优于纯推理或纯行动。

4 Reasoning Agent 的理论意义

4.1 推理作为无限维的内部动作

传统 Agent（无论 symbolic 还是 RL）的动作空间由环境固定定义（如上下左右），而 Reasoning Agent 引入了一种全新的动作类型——**推理**：

- 推理可以是任意长度的自然语言，动作空间是**无限的**
- 推理不改变外部环境，只改变 Agent 自身的上下文（内部状态）
- 推理影响后续的行动决策

4.2 三种 Agent 范式的对比

	Symbolic AI Agent	Deep RL Agent	Reasoning Agent
内部表示	符号状态	神经嵌入（向量）	自然语言
构建方式	手写规则	训练数百万步	Prompting / Fine-tuning
领域迁移	极差	差	好
表示维度	有限符号集	固定维度向量	无限语言空间

表 1: 三种 Agent 范式对比

自然语言作为中间表示的优势：

- 利用 LLM 预训练的丰富知识，无需大量额外训练

- 表达能力无限——可以思考一个词，也可以思考一百万个 token
- 天然支持 inference-time scaling

4.3 本章小结

Reasoning Agent 的本质创新在于用自然语言作为 Agent 的内部表示和推理媒介，这赋予了 Agent 无限维的内部动作空间和预训练知识的零样本迁移能力。

5 长期记忆 (Long-term Memory)

5.1 短期记忆的局限

LLM Agent 的“短期记忆”即上下文窗口 (context window)，存在三个根本限制：

1. **只能追加** (append-only)：只能不断添加新 token，不能编辑或删除
2. **容量有限**：即使有百万 token 窗口，仍有上限
3. **不持久**：任务结束或上下文清空后，所有“记忆”消失

金鱼困境

没有长期记忆的 Agent 就像金鱼——即使今天证明了黎曼猜想，明天也得从头再来。更严重的是，下次尝试不一定能成功。

5.2 Reflexion：最简单的长期记忆

Reflexion 是 ReAct 的自然扩展：

1. Agent 尝试解决任务（如编写代码）
2. 任务失败（如单元测试未通过）
3. Agent **反思**失败原因（如“我忘记处理边界情况”）
4. 将反思结果写入长期记忆
5. 下次尝试时读取长期记忆，避免重复错误

一种新型学习范式

Reflexion 本质上是一种**通过语言进行学习**的方法，区别于传统 RL 通过梯度下降更新权重：

- 不需要标量奖励信号，可以利用任何文本反馈（编译错误、测试结果、教师评语等）
- 不通过反向传播更新参数，而是通过更新文本记忆影响未来行为

5.3 更复杂的长期记忆形式

- **Voyager**: 在 Minecraft 中学习技能，将成功的代码（如“制作剑”）存入技能库，后续任务可直接调用。
- **Generative Agents**: 25 个模拟人类的 Agent 在小镇中生活，每个 Agent 维护一份事件日志（episodic memory）和从经验中提炼的语义记忆（semantic memory），这些记忆影响其后续行为。
- **Fine-tuning 作为长期记忆**: 也可以直接 fine-tune 模型参数，将经验“写入”神经网络权重。

5.4 KOALA: Agent 的统一抽象

Shunyu Yao 提出了 KOALA 框架，用三个组件统一描述任意 Agent:

1. **Memory**: 信息存储的位置（上下文窗口 + 外部存储 + 模型参数）
2. **Action Space**: Agent 可以执行的动作集合
3. **Decision-making Procedure**: 给定记忆和动作空间，如何选择下一步动作

5.5 本章小结

长期记忆使 Agent 从“无状态”的一次性求解器进化为能够积累经验、持续改进的学习系统。记忆可以是文本日志、技能代码库，甚至模型参数本身。

6 新兴应用领域

6.1 数字自动化 (Digital Automation)

LLM Agent 开启了全新的应用维度——**数字自动化**:

- **WebShop**: Agent 在模拟亚马逊网站上执行购物任务（搜索、比较、定制、购买），基于百万级真实商品数据构建。
- **网页交互**: Agent 操作真实网页完成各种任务。
- **SWE-bench**: Agent 给定 GitHub 仓库和 issue，需要理解代码库、编写测试、生成修复补丁——真正的软件工程任务。

6.2 科学发现

如 CHEM-AI 工作: Agent 分析化学数据、使用工具（Python、搜索引擎），提出新化合物方案，方案在湿实验室中被合成验证——Agent 的动作空间延伸到物理世界。

6.3 本章小结

LLM Agent 的应用已从传统 NLP/RL 的问答和游戏扩展到实际的数字自动化和科学发现，这是以往 Agent 范式无法企及的。

7 未来方向

Shunyu Yao 提出了五个关键研究方向：

7.1 训练 (Training)

模型-Agent 协同进化

当前 LLM 的训练数据主要来自互联网文本，缺乏 Agent 轨迹数据（思考过程、动作序列、环境反馈）。解决方案：用 prompted agent 生成大量轨迹数据，再用这些数据 fine-tune 模型，形成正向循环。类比 GPU 与深度学习的协同进化。

7.2 接口 (Interface)

- 人机接口 (HCI) 已研究数十年，但 Agent 的最优接口与人类不同
- 例如：人类搜索文件用 `ls + cd`（每次一个结果），但 Agent 更适合一次获取多个结果
- 不优化 Agent，而是优化**环境**——同一模型在更好的接口下表现更好

7.3 鲁棒性 (Robustness)

Pass@K vs. 鲁棒性

学术界关注 pass@K（K 次尝试中至少成功一次），但现实应用需要**鲁棒性**（K 次中每次都成功）。当前所有模型在采样次数增加时，鲁棒性单调下降。客服 Agent 失败一次就可能失去客户——不能仅追求“能做到”，还要“总能做到”。

7.4 人机协作 (Human-in-the-loop)

真实场景中 Agent 需要与人交互——客户不会一次给出所有信息，Agent 需要多轮对话获取所需信息。

7.5 评测基准 (Benchmarks)

- 需要更实际、更大规模的基准（如 Tau-Bench：客服场景）
- 需要评估鲁棒性而非仅评估能力上限
- 需要包含人机交互元素

7.6 本章小结

LLM Agent 领域的五个关键方向——训练、接口、鲁棒性、人机协作、评测——都有大量低垂果实，且不需要巨量 GPU 资源，适合学术界研究。

8 总结与延伸

8.1 核心要点

1. **Reasoning Agent** 是一种全新的 Agent 范式，以自然语言作为内部表示，具有无限维的推理空间。
2. **ReAct** 将推理和行动统一，两者相互增益：推理指导行动规划和异常处理，行动为推理提供外部信息和计算。
3. **长期记忆**（Reflexion、Voyager、Generative Agents）使 Agent 能跨任务积累经验。
4. **数字自动化**是 LLM Agent 开辟的全新应用维度。
5. 简洁性（simplicity）是最有力的研究品质——简单意味着通用。

8.2 拓展阅读

- Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR 2023.
- Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” NeurIPS 2023.
- Wang et al., “Voyager: An Open-Ended Embodied Agent with Large Language Models,” 2023.
- Park et al., “Generative Agents: Interactive Simulacra of Human Behavior,” UIST 2023.
- Yao et al., “WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents,” NeurIPS 2022.
- Zhou et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” ICLR 2024.
- Yao et al., “Tau-Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” 2024.